

Nexus — LinkedIn Redesign

Integration Test Specification

CS485 AI-Assisted Software Engineering — Spring 2026

Team: Krishi Shah, Dhyan Soni, Saanvi Elaty, Victor Jimenez

This document defines the English-language integration test specification for the Nexus application. Integration tests verify code pathways that span both the frontend (vanilla JS / React, served via Render) and the backend (Flask + PostgreSQL/SQLite REST API, hosted on Render). Each section identifies the functionality under test and provides a detailed test table.

1. Scope of Integration Testing

Integration tests are required for every code pathway that requires both the frontend API client (`js/api.js` → `window.API`) and the Flask backend (`backend/app.py`) to execute together. Unit tests that mock the database or isolate individual functions are out of scope here.

2. Functionality Under Test

The following frontend→backend code pathways must be covered by at least one integration test each:

2.1 Authentication

- User login with valid credentials (POST `/api/auth/login`)
- User login with invalid credentials — expect 401
- New user registration (POST `/api/auth/register`)
- Registration with duplicate email — expect 409
- Registration with missing/invalid fields — expect 400

2.2 Current User (Profile)

- Fetch current user profile (GET `/api/me`)
- Update profile fields — name, headline, location (PUT `/api/me`)
- Add education entry (POST `/api/me/education`)
- Add skill (POST `/api/me/skills`)
- Fetch profile readiness score (GET `/api/profile-readiness`)
- Fetch social state (GET `/api/me/social`)

2.3 Feed & Posts

- Fetch the post feed (GET `/api/feed`)
- Create a new post (POST `/api/feed`)
- Like a post (POST `/api/feed/:id/like`)
- Comment on a post (POST `/api/feed/:id/comments`)
- Delete a post (DELETE `/api/feed/:id`)

2.4 Connections & Network

- Send a connection request (POST `/api/me/connections/:id`)
- Accept a connection request (POST `/api/me/connections/:id/accept`)
- Follow/unfollow a user (POST `/api/me/following/:id`)

- Fetch all users for network page (GET /api/users)
- Fetch a specific user profile (GET /api/users/:id)

2.5 Messaging

- Fetch all conversations (GET /api/conversations)
- Fetch a single conversation with messages (GET /api/conversations/:id)
- Create a new conversation (POST /api/conversations)
- Send a message in a conversation (POST /api/conversations/:id/messages)

2.6 Jobs

- Fetch jobs list (GET /api/jobs)
- Fetch a single job (GET /api/jobs/:id)
- Save/unsave a job (POST /api/me/saved-jobs/:id)
- Apply to a job (POST /api/me/applied-jobs/:id)

2.7 Notifications

- Fetch notifications (GET /api/notifications)
- Mark a single notification as read (PATCH /api/notifications/:id/read)
- Mark all notifications as read (PATCH /api/notifications/read-all)

2.8 Events & Groups

- Fetch events list (GET /api/events)
- Create a new event (POST /api/events)
- Attend an event (POST /api/events/:id/attend)
- Fetch groups list (GET /api/groups)
- Create a new group (POST /api/groups)
- Join/leave a group (POST /api/me/groups/:id/toggle)

2.9 Search

- Search with a query string (GET /api/search?q=...)
- Search with empty string — expect 200 with results or empty list

2.10 Outreach — Story #1: Message Guidance

- Generate an outreach message with valid recipient and tone (POST /api/outreach/generate)
- Generate outreach with missing recipientId — expect 400
- Generate outreach with invalid recipientId — expect 404 or 400
- Generate outreach with all optional fields populated

2.11 Outreach — Story #2: Readiness Check

- Fetch outreach readiness for current user (GET /api/outreach/readiness)
- Fetch outreach readiness for a specific userId
- Fetch readiness for non-existent user — expect 404

3. Test Tables

Each row specifies: the test ID, purpose, HTTP method and endpoint called, the inputs sent, and the expected output if the test passes.

3.1 Authentication

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-A01	Login succeeds with valid credentials	POST /api/auth/login	email: "alex.johnson@gmail.com", password: "password123"	HTTP 200, body contains {user: {...}, token: "..."}
IT-A02	Login fails with wrong password	POST /api/auth/login	email: "alex.johnson@gmail.com", password: "wrongpass"	HTTP 401, body contains {error: ...}
IT-A03	Login fails with missing email	POST /api/auth/login	email: "", password: "password123"	HTTP 400
IT-A04	Register new user successfully	POST /api/auth/register	name: "Test User", email: unique generated email, password: "securepass123"	HTTP 201, body contains {user: {id, name, email}, token}
IT-A05	Register fails with duplicate email	POST /api/auth/register	name: "Alex J", email: "alex.johnson@gmail.com", password: "pass1234"	HTTP 409
IT-A06	Register fails with short password	POST /api/auth/register	name: "Bob", email: "bob@x.com", password: "short"	HTTP 400
IT-A07	Register fails with invalid email format	POST /api/auth/register	name: "Bob", email: "notanemail", password: "pass1234"	HTTP 400

3.2 Current User / Profile

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-U01	Fetch current user profile (authenticated)	GET /api/me	Authorization: Bearer <token>	HTTP 200, returns user object with id, name, headline
IT-U02	Unauthenticated request falls back to user id=1	GET /api/me	No Authorization header	HTTP 200, returns user with id=1 (Alex Johnson)
IT-U03	Update profile headline	PUT /api/me	Authorization header, body: {headline: "Software Engineer"}	HTTP 200, returned user has updated headline
IT-U04	Update profile with no valid fields returns 400	PUT /api/me	Authorization header, body: {invalidField: "x"}	HTTP 400
IT-U05	Add education entry	POST /api/me/education	Authorization header, body: {school: "MIT", degree: "BS", field: "CS", startDate: "2020", endDate: "2024"}	HTTP 200, returned user education array includes entry with school "MIT"
IT-U06	Add education without school returns 400	POST /api/me/education	Authorization header, body: {degree: "BS"}	HTTP 400
IT-U07	Add skill	POST /api/me/skills	Authorization header, body: {skill: "Python"}	HTTP 200, returned user skills array includes "Python"

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-U08	Fetch profile readiness score	GET /api/profile-readiness	Authorization header	HTTP 200, body contains {score: <number>}
IT-U09	Fetch social state	GET /api/me/social	Authorization header	HTTP 200, body contains {connections, following, savedJobs, appliedJobs}

3.3 Feed & Posts

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-F01	Fetch feed returns list of posts	GET /api/feed	Authorization header	HTTP 200, body is list of post objects each with id, content, author or authorId, and a likes count field
IT-F02	Create a new text post	POST /api/feed	Authorization header, body: {content: "Hello Nexus!", imageUrl: null}	HTTP 201, returned post has matching content and id
IT-F03	Create post without content returns 400	POST /api/feed	Authorization header, body: {}	HTTP 400
IT-F04	Like a post toggles like count	POST /api/feed/:id/like	Authorization header; freshly created post id (starts at 0 likes). Like endpoint called twice on the same post.	HTTP 200 on both calls. After first like: likeCount == 1 (verified via GET /api/feed). After second like: likeCount == 0.
IT-F05	Comment on a post	POST /api/feed/:id/comments	Authorization header, body: {text: "Great post!"}	HTTP 201, returned comment has text "Great post!"
IT-F06	Delete own post removes it from feed	DELETE /api/feed/:id	Authorization header, id of post owned by current user (freshly created)	HTTP 200; subsequent GET /api/feed does not contain deleted post id
IT-F07	Delete another user's post returns 403	DELETE /api/feed/:id	Authorization header, id of post owned by a different registered user	HTTP 403

3.4 Connections & Network

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-N01	Fetch all users returns list	GET /api/users	Authorization header	HTTP 200, body is a list of user objects
IT-N02	Fetch a specific user by id	GET /api/users/2	Authorization header	HTTP 200, body is user object with id=2
IT-N03	Fetch non-existent user returns 404	GET /api/users/99999	Authorization header	HTTP 404
IT-N04	Send a connection request	POST /api/me/connections/:id	Authorization header, target is a freshly registered user id	HTTP 200

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-N05	Accept a connection request	POST /api/me/connections/:id/accept	Authorization header; a secondary user first sends a connection request to the main user, then main user accepts	HTTP 200 on acceptance
IT-N06	Toggle follow on a user	POST /api/me/following/:id	Authorization header, freshly registered target user id	HTTP 200

3.5 Messaging

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-M01	Fetch all conversations	GET /api/conversations	Authorization header	HTTP 200, body is a list of conversation objects
IT-M02	Fetch single conversation with messages (requires conversation id=1 to exist in DB)	GET /api/conversations/1	Authorization header	HTTP 200, body contains conversation object with a messages array. Passes on deployed Render instance (seeded DB) and on a local DB that has been used previously. Fails on a completely fresh local database.
IT-M03	Create a new conversation	POST /api/conversations	Authorization header, body: {participantId: <freshly registered user id>}	HTTP 200 or 201, returned body contains conversation id
IT-M04	Send a message in a conversation	POST /api/conversations/:id/messages	Authorization header, conversation id from IT-M03, body: {text: "Hey there!"}	HTTP 201, returned message has text "Hey there!"
IT-M05	Send empty message returns 400	POST /api/conversations/:id/messages	Authorization header, valid conversation id, body: {text: ""}	HTTP 400

3.6 Jobs

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-J01	Fetch all jobs	GET /api/jobs	Authorization header	HTTP 200, body is list of job objects with id, title, and/or company
IT-J02	Fetch a specific job by id	GET /api/jobs/1	Authorization header	HTTP 200, body is job object with id=1
IT-J03	Save a job toggles saved state	POST /api/me/saved-jobs/1	Authorization header, job id=1	HTTP 200; GET /api/me/social before and after confirms job id=1 toggled in savedJobs array
IT-J04	Apply to a job	POST /api/me/applied-jobs/1	Authorization header, job id=1	HTTP 200; subsequent GET /api/me/social shows job id=1 in appliedJobs array

3.7 Notifications

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-NT01	Fetch notifications returns list	GET /api/notifications	Authorization header	HTTP 200, body is list of notification objects each with id and isRead field
IT-NT02	Mark a single notification as read	PATCH /api/notifications/:id/read	Authorization header; id is dynamically fetched from the first available notification	HTTP 200, returned notification has isRead: true
IT-NT03	Mark all notifications as read	PATCH /api/notifications/read-all	Authorization header. Skipped if no notifications exist.	HTTP 200; subsequent GET /api/notifications returns a non-empty list where all notifications have isRead: true

3.8 Events & Groups

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-E01	Fetch events list	GET /api/events	Authorization header	HTTP 200, body is list of event objects
IT-E02	Create a new event	POST /api/events	Authorization header, body: {name: "Hackathon", date: "2026-05-01", location: "Online", description: "Coding event"}	HTTP 201, returned event has name "Hackathon"
IT-E03	Attend an event	POST /api/events/:id/attend	Authorization header; event id is dynamically obtained by creating a new event first	HTTP 200
IT-G01	Fetch groups list (no auth required)	GET /api/groups	No Authorization header	HTTP 200, body is list of group objects
IT-G02	Create a new group	POST /api/groups	Authorization header, body: {name: "AI Enthusiasts", description: "A group for AI fans", industry: "Technology"}	HTTP 201, returned group has name "AI Enthusiasts"
IT-G03	Toggle group membership	POST /api/me/groups/:id/toggle	Authorization header; group id dynamically fetched from first available group	HTTP 200, group membership added or removed

3.9 Search

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-S01	Search for a known user by name	GET /api/search?q=Alex	Authorization header, q=Alex	HTTP 200, results.users includes a user with "Alex" in name
IT-S02	Search with empty query returns 200	GET /api/search?q=	Authorization header, q=	HTTP 200, returns results (all or empty, not an error)
IT-S03	Search for a job title keyword	GET /api/search?q=Engineer	Authorization header, q=Engineer	HTTP 200, results.jobs is a list (may include job postings matching Engineer)

3.10 Outreach — Story #1: Message Guidance

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-O01	Generate outreach message with valid inputs	POST /api/outreach/generate	Authorization header, body: {recipientId: 2, tone: "professional", goal: "networking", custom_note: "", details: {}}	HTTP 200, body contains draft field with a non-empty generated message string
IT-O02	Generate outreach with all optional fields populated	POST /api/outreach/generate	Authorization header, body: {recipientId: 2, tone: "casual", goal: "job_inquiry", custom_note: "Met at conference", details: {}}	HTTP 200, body contains draft field with a non-empty generated message string (unrecognised tone falls back to professional template)
IT-O03	Generate outreach missing recipientId returns 400	POST /api/outreach/generate	Authorization header, body: {tone: "professional"}	HTTP 400
IT-O04	Generate outreach with invalid recipientId	POST /api/outreach/generate	Authorization header, body: {recipientId: 999999, tone: "professional", goal: "networking"}	HTTP 404 or HTTP 400

3.11 Outreach — Story #2: Readiness Check

ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-R01	Fetch readiness score for current user	GET /api/outreach/readiness	Authorization header	HTTP 200, body contains {score: <number>, breakdown: [...]}
IT-R02	Fetch readiness score for a specific userId	GET /api/outreach/readiness?userId=2	Authorization header, userId=2	HTTP 200, body contains score reflecting user 2's profile completeness
IT-R03	Fetch readiness for non-existent user returns 404	GET /api/outreach/readiness?userId=999999	Authorization header, userId=999999	HTTP 404

4. Environment Configuration

Tests are parameterized by environment variables so they run against both localhost and the deployed Render instance without code changes.

Variable	Localhost Value	Deployed Value
BASE_URL	http://localhost:5000/api	https://linkedin-redesign-z364.onrender.com/api
TEST_EMAIL	alex.johnson@gmail.com	alex.johnson@gmail.com
TEST_PASSWORD	password123	password123

Note on IT-M02: This test fetches conversation id=1 directly. It passes on the deployed Render instance (which has seeded data) and on any local database that has been used for testing previously (since prior runs of IT-M03 create conversations that persist in the SQLite file). It will fail on a completely fresh local database with no prior data.

In PowerShell, use `$env:` syntax to set environment variables before running:

```
$env:BASE_URL = "http://localhost:5000/api"
pytest backend/test_integration.py -v

$env:BASE_URL = "https://linkedin-redesign-z364.onrender.com/api"
pytest backend/test_integration.py -v
```

5. Test Framework & Tooling

Integration tests are implemented in Python using **pytest** and the **requests** library. Each test calls the REST API directly, reproducing exactly what `js/api.js` does from the browser.

Why direct API calls constitute valid frontend+backend integration testing

The frontend layer of this application (`js/api.js`) contains no business logic. Every exported method is a thin wrapper around a single `fetch()` call — for example:

```
getFeed: () => request('GET', '/feed'),
likePost: (id) => request('POST', `/feed/${id}/like`),
deletePost:(id) => request('DELETE', `/feed/${id}`),
```

The only work the frontend performs is constructing the HTTP request (method, path, Authorization header, JSON body) and reading the HTTP response (status code, JSON body). Our pytest tests reproduce that work identically — same methods, same paths, same headers, same payloads — meaning they exercise the complete frontend→backend integration boundary without requiring a browser process.

Running a headless browser via Selenium or Playwright would execute the same HTTP calls with additional overhead and flakiness from DOM rendering, but would not exercise any additional code paths, because there are no code paths between the UI event and the network call that are worth integration-testing separately (those are covered by the unit tests).

Additional note: `js/api.js` hardcodes the production Render URL as BASE, so when running the app locally the frontend already targets the deployed backend. Our integration tests improve on this by parameterising the base URL via the `BASE_URL` environment variable, allowing tests to target either environment independently.

- **pytest** — test runner and assertion framework
- **requests** — HTTP client for firing real API calls
- **os.environ** — for `BASE_URL` / `TEST_EMAIL` / `TEST_PASSWORD` configuration
- **confest.py** — shared fixtures: `auth_token`, `auth_headers`, `base_url`
- **uuid** — generates unique emails for registration tests to avoid duplicate-email conflicts across repeated runs

6. Test Summary

Section	Area	# Tests
3.1	Authentication	7
3.2	Current User / Profile	9
3.3	Feed & Posts	7
3.4	Connections & Network	6
3.5	Messaging	5
3.6	Jobs	4
3.7	Notifications	3

Section	Area	# Tests
3.8	Events & Groups	6
3.9	Search	3
3.10	Outreach — Story #1	4
3.11	Outreach — Story #2	3
	TOTAL	57